23CSE211

Design and Analysis of Algorithms

Lab-4 26/12/2025

Submitted to

Dr SS Chakravarthi

Submitted by

Ruhan T

CH.SC.U4CSE24240

LAB-4

Prim's & Kruskal's Algorithm

Prim's Algorithm

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define V 7
int minKey(int key[], bool mstSet[]) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++)
        if (mstSet[v] == false && key[v] < min)
            min = key[v], min_index = v;
    return min_index;
}
void printMST(int parent[], int graph[V][V]) {
    printf("Prim's Algorithm MST:\n");
    printf("Edge \tWeight\n");
    int totalWeight = 0;
    for (int i = 1; i < V; i++) {
        printf("%c - %c \t%d \n", parent[i] + 'A', i + 'A', graph[i][parent[i]]);
        totalWeight += graph[i][parent[i]];
    }
    printf("Total Minimum Weight: %d\n", totalWeight);
}
void primMST(int graph[V][V]) {
    int parent[V];
    int key[V];
    bool mstSet[V];
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = true;
        for (int v = 0; v < V; v++)
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
```

```
    }
    printMST(parent, graph);
}
int main() {
    int graph[V][V] = {
        { 0, 12,  7,  5,  0,  0,  0 },
        {12,  0,  4,  0,  7,  0,  0 },
        { 7,  4,  0,  9,  3,  4,  0 },
        { 5,  0,  9,  0,  0,  7,  0 },
        { 0,  7,  3,  0,  0,  2,  2 },
        { 0,  0,  4,  7,  2,  0,  5 },
        { 0,  0,  0,  0,  2,  5,  0 }
    };
    primMST(graph);
    return 0;
}
```

Kruskal's Algorithm

```
#include <stdio.h>
#include <stdlib.h>
struct Edge {
    int src, dest, weight;
};
struct Graph {
    int V, E;
    struct Edge* edge;
};
struct subset {
    int parent;
    int rank;
};
struct Graph* createGraph(int V, int E) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->E = E;
    graph->edge = (struct Edge*)malloc(graph->E * sizeof(struct Edge));
    return graph;
}
int find(struct subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}
```

```
void Union(struct subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);
    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;
    else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}
int myComp(const void* a, const void* b) {
    struct Edge* a1 = (struct Edge*)a;
    struct Edge* b1 = (struct Edge*)b;
    return a1->weight > b1->weight;
}
void KruskalMST(struct Graph* graph) {
    int V = graph->V;
    struct Edge result[V];
    int e = 0;
    int i = 0;
    qsort(graph->edge, graph->E, sizeof(graph->edge[0]), myComp);
    struct subset* subsets = (struct subset*)malloc(V * sizeof(struct
subset));
    for (int v = 0; v < V; ++v) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }
    while (e < V - 1 && i < graph->E) {
        struct Edge next_edge = graph->edge[i++];
        int x = find(subsets, next_edge.src);
        int y = find(subsets, next_edge.dest);
        if (x != y) {
            result[e++] = next_edge;
            Union(subsets, x, y);
        }
    }
    printf("Kruskal's Algorithm MST:\n");
    printf("Edge \tWeight\n");
    int totalWeight = 0;
    for (i = 0; i < e; ++i) {
        printf("%c - %c \t%d \n",
```

```
                result[i].src + 'A',
                result[i].dest + 'A',
                result[i].weight);
            totalWeight += result[i].weight;
        }
        printf("Total Minimum Weight: %d\n", totalWeight);
        free(subsets);
    }
    int main() {
        int V = 7;
        int E = 12;
        struct Graph* graph = createGraph(V, E);
        graph->edge[0].src = 0; graph->edge[0].dest = 1; graph-
    >edge[0].weight = 12;
        graph->edge[1].src = 0; graph->edge[1].dest = 2; graph-
    >edge[1].weight = 7;
        graph->edge[2].src = 0; graph->edge[2].dest = 3; graph-
    >edge[2].weight = 5;
        graph->edge[3].src = 1; graph->edge[3].dest = 2; graph-
    >edge[3].weight = 4;
        graph->edge[4].src = 1; graph->edge[4].dest = 4; graph-
    >edge[4].weight = 7;
        graph->edge[5].src = 2; graph->edge[5].dest = 3; graph-
    >edge[5].weight = 9;
        graph->edge[6].src = 2; graph->edge[6].dest = 4; graph-
    >edge[6].weight = 3;
        graph->edge[7].src = 2; graph->edge[7].dest = 5; graph-
    >edge[7].weight = 4;
        graph->edge[8].src = 3; graph->edge[8].dest = 5; graph-
    >edge[8].weight = 7;
        graph->edge[9].src = 4; graph->edge[9].dest = 5; graph-
    >edge[9].weight = 2;
        graph->edge[10].src = 4; graph->edge[10].dest = 6; graph-
    >edge[10].weight = 2;
        graph->edge[11].src = 5; graph->edge[11].dest = 6; graph-
    >edge[11].weight = 5;
        KruskalMST(graph);
        return 0;
    }
```

```
ruhaaa@Master:~/Documents/Linux$ nano prim.c
ruhaaa@Master:~/Documents/Linux$ gcc prim.c -o prim
ruhaaa@Master:~/Documents/Linux$ ./prim
Edge    Weight
C - B   4
A - C   7
A - D   5
C - E   3
E - F   2
E - G   2
-----------------------
Total Weight: 23
ruhaaa@Master:~/Documents/Linux$ nano kruskal.c
ruhaaa@Master:~/Documents/Linux$ gcc kruskal.c -o kruskal
ruhaaa@Master:~/Documents/Linux$ ./kruskal
Kruskal's Algorithm MST:
Edge    Weight
E - F   2
E - G   2
C - E   3
B - C   4
A - D   5
A - C   7
Total Minimum Weight: 23
ruhaaa@Master:~/Documents/Linux$
```